

8.2.55

BALU-ai25btech11017

September 29, 2025

Question

Equation of the hyperbola with eccentricity $\frac{3}{2}$ and foci at $(\pm 2, 0)$ is

_____.

Theoretical Solution

Let us solve the given equation theoretically and then verify the solution computationally

According to the question,

$$e = \frac{3}{2} \mathbf{F}_1 = \begin{pmatrix} 2 \\ 0 \end{pmatrix} \quad \mathbf{F}_2 = \begin{pmatrix} -2 \\ 0 \end{pmatrix} \quad (1)$$

$$\mathbf{n} = \mathbf{F}_1 - \mathbf{F}_2 = \begin{pmatrix} 4 \\ 0 \end{pmatrix} \quad (2)$$

$$\mathbf{n}^T \mathbf{x} = c \quad (3)$$

hyperbola equation is

$$\mathbf{x}^T \mathbf{v} \mathbf{x} + 2\mathbf{u}^T \mathbf{x} + f = 0 \quad (4)$$

$$\mathbf{v} = \|\mathbf{n}\|^2 \mathbf{I} - e^2 \mathbf{n} \mathbf{n}^T \quad (5)$$

$$\mathbf{u} = ce^2 \mathbf{e}_1 - \mathbf{F} \quad (6)$$

$$f = \|\mathbf{n}\|^2 \|\mathbf{F}\|^2 - c^2 e^2 \quad (7)$$

$$\mathbf{u} = \frac{\mathbf{F}_1 + \mathbf{F}_2}{2} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad (8)$$

$$\mathbf{v} = \begin{pmatrix} -20 & 0 \\ 0 & 16 \end{pmatrix} \quad (9)$$

$$f = 60 \quad (10)$$

hence the required hyperbola equation is

$$\mathbf{x}^T \begin{pmatrix} -20 & 0 \\ 0 & 16 \end{pmatrix} \mathbf{x} + 60 = 0 \quad (11)$$

C Code

```
#include <stdio.h>
#include <math.h>

int main() {
    // Given
    double e = 1.5; // eccentricity
    double c = 2.0; // focus distance

    // Compute parameters
    double a = c / e; // semi-major axis
    double b2 = c*c - a*a; // b^2
    double b = sqrt(b2);

    // Conic form coefficients (for Python V, u, f)
    double V11 = 9.0/16.0;
    double V22 = -9.0/20.0;
    double f = -1.0;
```

```
// Output in Python-readable format
printf("# Hyperbola parameters\n");
printf("a = %.6f\n", a);
printf("b = %.6f\n", b);
printf("V = [[%.6f, 0], [0, %.6f]]\n", V11, V22);
printf("u = [[0], [0]]\n");
printf("f = %.1f\n", f);

return 0;
}
```

```
#Program to plot a hyperbola
#Code by GVV Sharma (Modified)
#Updated: Sep 2025

import numpy as np
import matplotlib.pyplot as plt
from numpy import linalg as LA
import sys

# For path to external scripts
sys.path.insert(0, '/sdcard/github/matgeo/codes/CoordGeo')

# Local imports
from line.funcs import *
from triangle.funcs import *
from conics.funcs import *
```

```
# Plotting setup
fig = plt.figure()
ax = fig.add_subplot(111, aspect='equal')
len = 100
y = np.linspace(-2, 2, len)

#-----
# Hyperbola Parameters from Question
# Eccentricity  $e = 3/2$ 
# Foci at (2, 0)  $c = 2$   $a = 4/3$ ,  $b^2 = c^2 - a^2 = 20/9$ 

a = 4 / 3
b = np.sqrt(20 / 9)

# Conic form:  $(x^2 / a^2) - (y^2 / b^2) = 1$ 
# Multiply both sides to get general quadratic:
#  $(9x^2 / 16) - (9y^2 / 20) = 1$ 
# V matrix, u vector, f scalar such that  $x^T V x + f = 0$ 
```


python Code

```
V = np.array([[9/16, 0], [0, -9/20]])
u = np.array([[0], [0]])
f = -1

# Get conic parameters using your utility function
n, c, F, O, lam, P, e = conic_param(V, u, f)

# Affine transform matrix (diagonal since axes aligned)
ParamMatrix = np.diag([a, b])

# Generate standard hyperbola
x = hyper_gen(y)
xStandardHyperLeft = np.block([[ -x], [y]])
xStandardHyperRight = np.block([[ x], [y]])

# Apply affine transform
Of = O.flatten()
xActualHyperLeft = P @ ParamMatrix @ xStandardHyperLeft + Of[:,
    np.newaxis]
```

```
xActualHyperRight = P @ ParamMatrix @ xStandardHyperRight + Of[:,  
    np.newaxis]  
  
# Directrix and Latus Rectum lines  
k1 = -2  
k2 = 2  
c1 = (n.T @ F).flatten()  
  
x_A = line_norm(n, c[0], k1, k2) # Left directrix  
x_B = line_norm(n, c1[0], k1, k2) # Left latus rectum  
x_C = line_norm(n, c[1], k1, k2) # Right directrix  
x_D = line_norm(n, c1[1], k1, k2) # Right latus rectum  
  
# Plotting  
plt.plot(xActualHyperLeft[0, :], xActualHyperLeft[1, :], label='  
    Hyperbola', color='r')  
plt.plot(xActualHyperRight[0, :], xActualHyperRight[1, :], color=  
    'r')
```

```
plt.plot(x_A[0, :], x_A[1, :], label='Directrix')
plt.plot(x_B[0, :], x_B[1, :], label='Latus Rectum')
plt.plot(x_C[0, :], x_C[1, :])
plt.plot(x_D[0, :], x_D[1, :])

# Plot origin and foci
tri_coords = np.block([0, F])
plt.scatter(tri_coords[0, :], tri_coords[1, :], c=np.arange(1, 4)
            )
vert_labels = ['$\mathbf{O}$', '$\mathbf{F}_1$', '$\mathbf{F}_2$']
for i, txt in enumerate(vert_labels):
    plt.annotate(txt, (tri_coords[0, i], tri_coords[1, i]),
                 textcoords="offset points", xytext=(-10, 5), ha='center')
```

```
# Axes formatting
ax = plt.gca()
ax.spines['top'].set_color('none')
ax.spines['left'].set_position('zero')
ax.spines['right'].set_color('none')
ax.spines['bottom'].set_position('zero')
plt.xlabel('$x$')
plt.ylabel('$y$')
plt.legend(loc='best')
plt.grid()
plt.axis('equal')

# For Termux
plt.savefig('chapters/11/11/4/13/figs/fig.pdf')
import subprocess
import shlex
subprocess.run(shlex.split("termux-open chapters/11/11/4/13/figs/
    fig.pdf"))

# Otherwise uncomment to show plot
# plt.show()
```

Plot

